Microsoft AI Engineer Program via Simplilearn

Submitted by Ronald M. Paraison

Mercedes-Benz Greener Manufacturing

Course-end Project 1

Problem: Reduce the time a Mercedes-Benz spends on the test bench by predicting `y` (testing time) from anonymized features representing permutations of car configuration.

Required steps:

1. Remove column(s) with zero variance.
2. Check for null and unique values for test and train sets.
3. Apply label encoder.
4. Perform dimensionality reduction.
5. Predict `test_df` values using XGBoost.

In [1]:
```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
import xgboost as xgb

pd.set_option('display.max_columns', 50)
```

1. Load the data

In [2]:
```
DATA_DIR = '/Users/paraison/Documents/MS AI Engineer Program/Machine Learning using Python/Dataset/CEP_Datasets/Mercedes Benz Greener Manufacturing Datasets'

train_df = pd.read_csv(f'{DATA_DIR}/train.csv')
test_df = pd.read_csv(f'{DATA_DIR}/test.csv')

print('train_df shape:', train_df.shape)
print('test_df shape:', test_df.shape)
train_df.head()
```

```
train_df shape: (4209, 378)
test_df shape: (4209, 377)
```

Out[2]:

	ID	y	X0	X1	X2	X3	X4	X5	X6	X8	X10	X11	X12	X13	X14	X15	X16	X17	X18
0	0	130.81	k	v	at	a	d	u	j	o	0	0	0	1	0	0	0	0	1
1	6	88.53	k	t	av	e	d	y	l	o	0	0	0	0	0	0	0	0	1
2	7	76.26	az	w	n	c	d	x	j	x	0	0	0	0	0	0	0	1	0
3	9	80.62	az	t	n	f	d	x	l	e	0	0	0	0	0	0	0	0	0
4	13	78.02	az	v	n	f	d	h	d	n	0	0	0	0	0	0	0	0	0

5 rows × 378 columns

2. Check for null and unique values (test and train sets)

In [3]:
```
print('Null values in train_df:', train_df.isnull().sum().sum())
print('Null values in test_df:', test_df.isnull().sum().sum())
```

```
Null values in train_df: 0
Null values in test_df: 0
```

In [4]:
```
# Unique value counts per column (train)
unique_counts_train = train_df.nunique().sort_values()
unique_counts_train.head(10)
```

Out[4]:
```
X290    1
X235    1
X11     1
X297    1
X107    1
X233    1
X268    1
X347    1
X93     1
X330    1
dtype: int64
```

In [5]:
```
# Unique value counts per column (test)
unique_counts_test = test_df.nunique().sort_values()
unique_counts_test.head(10)
```

Out[5]:
```
X258    1
X295    1
X296    1
X257    1
X369    1
X195    2
X262    2
X261    2
X260    2
X259    2
dtype: int64
```

3. Remove columns with zero variance

A column has zero variance when it takes only a single unique value across all rows — it carries no information for modeling, so it is dropped. `ID` and `y` are excluded from this check since they are the identifier and target, not features.

In [6]:
```
feature_cols = [c for c in train_df.columns if c not in ('ID', 'y')]

zero_variance_cols = [c for c in feature_cols if train_df[c].nunique()
== 1]
print(f'Zero-variance columns found ({len(zero_variance_cols)}):')
print(zero_variance_cols)

train_df.drop(columns=zero_variance_cols, inplace=True)
test_df.drop(columns=[c for c in zero_variance_cols if c in test_df.co
lumns], inplace=True)

print('\ntrain_df shape after drop:', train_df.shape)
print('test_df shape after drop:', test_df.shape)
```

```
Zero-variance columns found (12):
['X11', 'X93', 'X107', 'X233', 'X235', 'X268', 'X289', 'X290', 'X293',
'X297', 'X330', 'X347']

train_df shape after drop: (4209, 366)
test_df shape after drop: (4209, 365)
```

4. Apply label encoder

Columns `X0` - `X8` are categorical (object dtype); encode them to integers with `LabelEncoder`, fitting on the union of train + test categories so unseen test categories don't break the transform.

In [7]:
```
categorical_cols = train_df.select_dtypes(include='object').columns.to
list()
print('Categorical columns:', categorical_cols)

for col in categorical_cols:
    le = LabelEncoder()
    combined = pd.concat([train_df[col], test_df[col]], axis=0).astype
(str)
    le.fit(combined)
    train_df[col] = le.transform(train_df[col].astype(str))
    test_df[col] = le.transform(test_df[col].astype(str))

train_df[categorical_cols].head()
```

```
Categorical columns: ['X0', 'X1', 'X2', 'X3', 'X4', 'X5', 'X6', 'X8']
```

Out[7]:

	X0	X1	X2	X3	X4	X5	X6	X8
0	37	23	20	0	3	27	9	14
1	37	21	22	4	3	31	11	14
2	24	24	38	2	3	30	9	23
3	24	21	38	5	3	30	11	4
4	24	23	38	5	3	14	3	13

5. Perform dimensionality reduction

With ~360 remaining binary/encoded features, PCA reduces the feature space while retaining most of the variance.

In [8]:
```
X = train_df.drop(columns=['ID', 'y'])
y = train_df['y']
X_test_final = test_df.drop(columns=['ID'])

n_components = 0.95  # retain 95% of variance
pca = PCA(n_components=n_components, random_state=42)
X_pca = pca.fit_transform(X)
X_test_pca = pca.transform(X_test_final)

print(f'Original feature count: {X.shape[1]}')
print(f'Components retained for 95% variance: {pca.n_components_}')
print(f'X_pca shape: {X_pca.shape}')
print(f'X_test_pca shape: {X_test_pca.shape}')
```

```
Original feature count: 364
Components retained for 95% variance: 6
X_pca shape: (4209, 6)
X_test_pca shape: (4209, 6)
```

6. Predict test_df values using XGBoost

Hold out a validation split from the training data to sanity-check the model, then refit on all training data and predict on `test_df`.

In [9]:
```
X_train, X_val, y_train, y_val = train_test_split(X_pca, y, test_size=
0.2, random_state=42)

xgb_model = xgb.XGBRegressor(
    n_estimators=500,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1,
)

xgb_model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    verbose=False,
)

val_preds = xgb_model.predict(X_val)
print('Validation R2:', r2_score(y_val, val_preds))
print('Validation RMSE:', mean_squared_error(y_val, val_preds) ** 0.5)
```

```
Validation R2: 0.38689532799288473
Validation RMSE: 9.76881288558893
```

In [10]:
```
# Refit on the full training set, then predict on test_df
xgb_model_final = xgb.XGBRegressor(
    n_estimators=500,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1,
)
xgb_model_final.fit(X_pca, y)

test_predictions = xgb_model_final.predict(X_test_pca)

submission = pd.DataFrame({'ID': test_df['ID'], 'y': test_prediction
s})
submission.head()
```

Out[10]:

	ID	y
0	1	74.431999
1	2	94.872414
2	3	88.027802
3	4	81.935944
4	5	101.704208

In [11]:
```
output_path = f'{DATA_DIR}/mercedes_benz_test_predictions.csv'
submission.to_csv(output_path, index=False)
print('Saved predictions to:', output_path)
```

```
Saved predictions to: /Users/paraison/Documents/MS AI Engineer Progra
m/Machine Learning using Python/Dataset/CEP_Datasets/Mercedes Benz Gre
ener Manufacturing Datasets/mercedes_benz_test_predictions.csv
```